

PROJECT ARGUS

Plateforme SecOps agentique

Identité · Messagerie · Renseignement sur les menaces

Cahier des charges technique — Version 2.0

En une phrase

Une plateforme où un agent IA mène seul le triage d'un incident de sécurité : il interroge l'identité (Entra ID), la messagerie (SPF, DKIM, DMARC) et le renseignement sur les menaces (VirusTotal, AbuseIPDB, GreyNoise), puis rend en moins de 60 secondes un verdict argumenté dans une console web, avec un plan de remédiation soumis à validation humaine.

Ce document spécifie la suite du serveur MCP Entra ID SecOps livré en Phase 1. Ce dernier n'est pas remplacé : il devient le premier des quatre serveurs d'outils de la plateforme.

Rubrique	Valeur
Nom de code	ARGUS — Agentic Response & Guided Understanding System
Nature	Plateforme web + agent d'orchestration + 4 serveurs MCP
Point de départ	Serveur MCP Entra ID SecOps (Phase 1), livré et opérationnel
Ajouts majeurs	Console web analyste · sécurité de la messagerie · renseignement sur les menaces
Durée cible	6 sprints — environ 14 semaines à temps partiel
Livrable final	Plateforme déployée + jeu d'évaluation + démo vidéo + dossier technique

Sommaire

#	Section	#	Section
1	Ce qui existe, et ce qui manque	8	L'agent d'orchestration
2	La cible : une plateforme, pas trois outils	9	Évaluation, observabilité, coûts
3	Architecture en quatre plans	10	Sécurité de la plateforme
4	Serveur MCP — Renseignement sur les menaces	11	Stack technique
5	Serveur MCP — Sécurité de la messagerie	12	Feuille de route en 6 sprints
6	Serveur MCP — Dossier et remédiation	13	Les 15 signaux de niveau senior
7	La console web analyste	A	Annexe — Questions d'entretien

1. Ce qui existe, et ce qui manque

1.1 L'acquis. La Phase 1 a livré un serveur MCP fonctionnel, conteneurisé et connecté à Microsoft Graph : six outils en lecture seule, 96 tests, intégration continue verte, troncature agressive des réponses, et agrégats calculés en Python plutôt que déduits par le modèle. C'est une fondation solide, et elle est terminée.

1.2 La limite. Un serveur MCP ne fait rien tout seul. Il attend qu'un humain ouvre un client de discussion et pose une question. Trois manques en découlent :

- **Un seul domaine.** L'identité ne suffit pas à trancher. Une adresse IP suspecte demande un enrichissement externe ; un incident de phishing demande l'analyse des en-têtes de messagerie.
- **Aucune interface.** Le projet n'existe que dans un terminal ou dans une fenêtre de discussion. Il n'est ni montrable à une équipe, ni utilisable par un analyste qui n'a pas installé Claude Desktop.
- **Aucune mémoire.** Chaque question repart de zéro. Rien n'est conservé, rien n'est mesuré, rien n'est auditable six semaines plus tard.

Une observation déterminante

Trois projets tournent déjà sur le poste de développement : le serveur MCP Entra ID, un tableau de bord DMARC, et un scanner de vulnérabilités web. Ce ne sont pas trois projets séparés : ce sont les briques d'une même plateforme. Les réunir sous une orchestration commune produit un travail bien plus convaincant que trois dépôts juxtaposés — et cela ne demande pas de tout réécrire.

2. La cible : une plateforme, pas trois outils

Un analyste reçoit une alerte. Aujourd'hui, il ouvre le portail Entra, puis VirusTotal, puis son outil DMARC, puis son système de tickets. Il recopie des adresses IP d'une fenêtre à l'autre et reconstitue une chronologie à la main. Vingt minutes par alerte, trente fois par jour.

ARGUS remplace cet enchaînement manuel par une seule question posée dans une console web. L'agent choisit lui-même les outils, croise les domaines, et rend un dossier instruit.

Domaine	Question traitée	Origine
Identité	Qui est ce compte, que s'est-il passé à la connexion, l'attaquant a-t-il modifié quelque chose ?	Existant — Phase 1
Renseignemen t	Cette adresse IP, ce domaine, ce fichier sont-ils connus comme malveillants ?	À construire
Messagerie	Ce courriel est-il usurpé ? Notre domaine est-il protégé contre l'usurpation ?	À construire
Dossier	Que retient-on de l'incident, et quelles actions propose-t-on ?	À construire

2.1 Pourquoi la messagerie change la donne. La majorité des compromissions de compte commencent par un courriel. Or SPF, DKIM et DMARC sont des mécanismes que peu de développeurs maîtrisent, et dont l'analyse est entièrement automatisable. C'est un domaine où un outil bien fait apporte une valeur immédiate — et où le tableau de bord DMARC déjà écrit trouve sa place naturelle.

3. Architecture en quatre plans

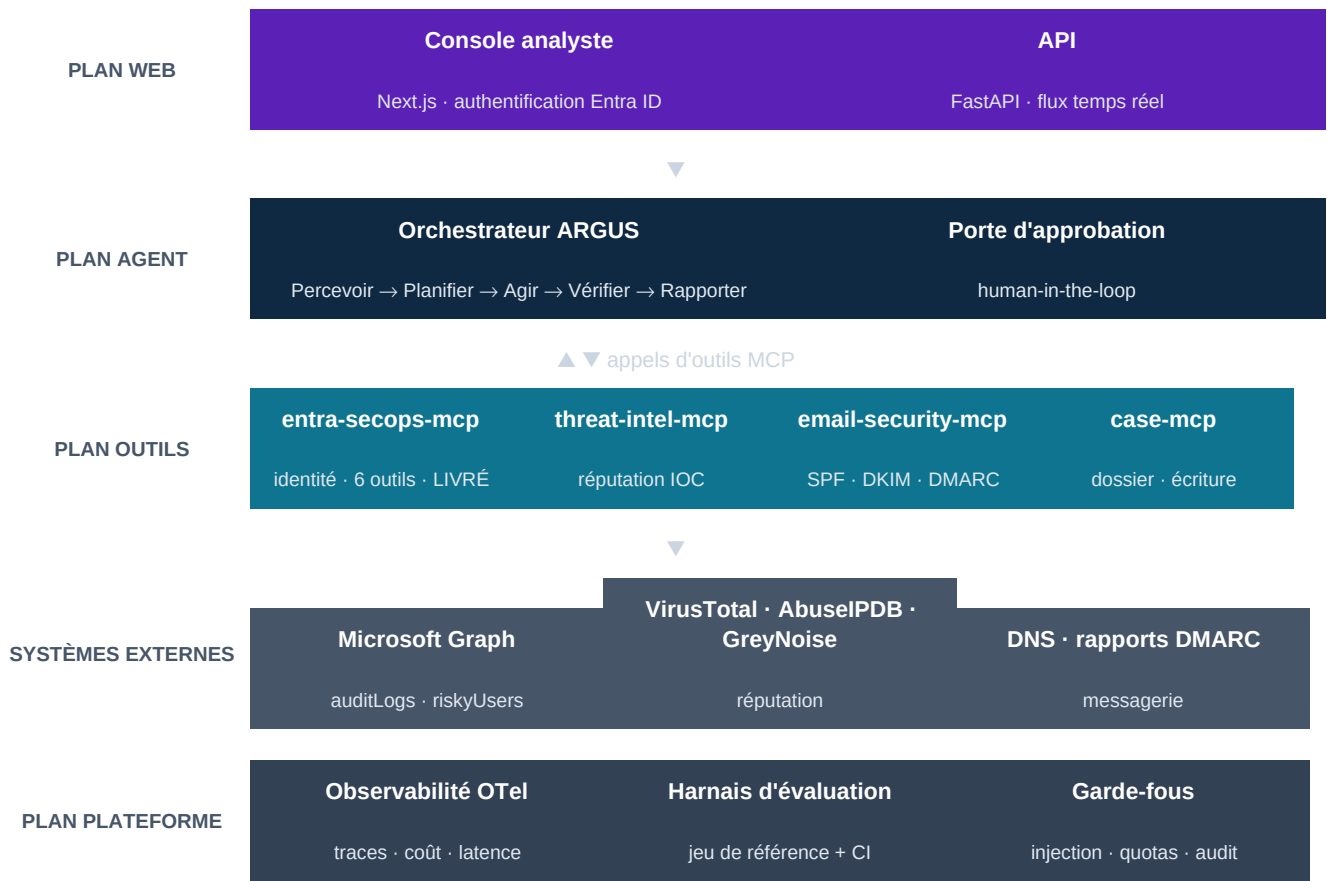


Figure 1 — Les flèches sont des appels d'outils MCP, pas des dépendances de code. Chaque serveur reste un processus autonome, conteneurisé et utilisable seul depuis un client comme Claude Desktop.

3.1 Pourquoi quatre serveurs et non un seul.

- **Cloisonnement des privilèges.** Seul case-mcp détient des droits d'écriture, et il est le seul soumis à la porte d'approbation. L'agent ne peut pas désactiver un compte, même s'il le décidait.
- **Rayon d'explosion réduit.** Une compromission du serveur de renseignement, qui parle à des API publiques, n'ouvre aucun accès au tenant Entra.
- **Clés séparées.** La clé VirusTotal et le secret Entra ne vivent pas dans le même processus.
- **Réutilisabilité.** Chaque serveur reste utile seul — y compris pour un collègue qui ne veut que l'enrichissement d'IOC.

4. Serveur MCP — Renseignement sur les menaces

C'est ici qu'entrent VirusTotal et ses équivalents. L'écueil serait d'écrire un simple relais d'API : il en existe des dizaines en open source, et cela n'apporterait rien. L'ingénierie tient en trois points : agrégation multi-sources, normalisation en un verdict unique, et mise en cache.

Outil	Sources interrogées	Sortie normalisée
enrich_ip	VirusTotal, AbuseIPDB, GreyNoise	verdict, score 0-100, ASN, pays, hébergeur, indicateur VPN / Tor / proxy, activité de scan connue
enrich_domain	VirusTotal, urlscan.io	verdict, âge du domaine, enregistrements DNS, catégorisation
enrich_url	VirusTotal, urlscan.io	verdict, redirections, capture d'écran, ressemblance avec un domaine légitime

Outil	Sources interrogées	Sortie normalisée
enrich_file_hash	VirusTotal	verdict, ratio de détections, familles de maliciels nommées, date de première observation
bulk_enrich	Toutes, en parallèle	Enrichit jusqu'à 20 indicateurs en un seul appel — évite à l'agent 20 allers-retours

4.1 Le point technique qui fait la différence. Les trois sources renvoient trois échelles incomparables : VirusTotal un ratio de moteurs, AbuseIPDB un score de confiance sur 100, GreyNoise une classification qualitative. Un relais laisserait le modèle arbitrer — c'est-à-dire deviner. ARGUS applique une fusion déterministe, écrite en Python et couverte par des tests.

```
def fuse(vt, abuse, greynoise) -> IpVerdict:
    # GreyNoise 'benign' prime : scanner legitime connu (Shodan, Censys).
    # Sans cette regle, les scanners de recherche declenchent des
    # faux positifs a repetition et l'outil perd la confiance de l'equipe.
    if greynoise.classification == 'benign':
        return IpVerdict('benign', 0, 'Scanner Internet legitime reference')

    score = max(
        vt.malicious / max(vt.total_engines, 1) * 100,
        abuse.confidence_score,
    )
    if greynoise.classification == 'malicious':
        score = min(100, score + 25)

    level = ('malicious' if score >= 70 else
            'suspicious' if score >= 30 else 'benign')
    return IpVerdict(level, round(score), _explain(vt, abuse, greynoise))
```

Extrait 1 — La logique de décision reste dans du code testé, jamais dans le prompt. C'est le principe le plus important de la conception d'agents fiables : aucun texte injecté par un attaquant ne peut renverser un score calculé en Python.

Quotas : une contrainte de conception, pas un détail

Le palier gratuit de VirusTotal est de l'ordre de quelques requêtes par minute et de quelques centaines par jour. Un agent qui enrichit naïvement chaque IP de chaque journal épuise ce quota en une seule investigation. Trois mécanismes sont donc obligatoires : un **cache Redis de 24 h** par indicateur, un **limiteur de débit** côté serveur, et un **plafond d'indicateurs** par dossier. Les mêmes adresses IP reviennent sans cesse : le cache évite la grande majorité des appels.

5. Serveur MCP — Sécurité de la messagerie

La majorité des compromissions de compte commencent par un courriel. Ce serveur répond à deux familles de questions : *ce message est-il usurpé ?* et *notre domaine est-il protégé contre l'usurpation ?*

Outil	Objet	Ce qui est vérifié
check_spf	Analyse l'enregistrement SPF d'un domaine	Présence, mécanisme final (-all strict ou ~all permissif), et surtout le nombre de résolutions DNS : au-delà de 10, la norme impose un échec et SPF cesse silencieusement de protéger
check_dkim	Vérifie les clés DKIM publiées	Présence du sélecteur, longueur de clé (1024 bits reste courant mais 2048 est attendu), clé révoquée

Outil	Objet	Ce qui est vérifié
check_dmarc	Analyse la politique DMARC	Politique p=none (observation seule, donc sans protection), quarantaine ou reject ; pourcentage appliqué, adresses de rapport, alignement strict ou relaxé
analyze_email_headers	Décompose les en-têtes d'un message reçu	Résultats SPF / DKIM / DMARC, alignement du domaine From avec le domaine authentifié, chemin des serveurs relais, incohérences de Reply-To
parse_dmarc_report	Lit un rapport agrégé DMARC (XML)	Volumes par source, taux de réussite, expéditeurs légitimes non alignés, sources d'usurpation
check_domain_posture	Synthèse de la posture d'un domaine	SPF + DKIM + DMARC + MTA-STS, avec une note et les écarts classés par gravité

Le point que peu de candidats connaissent

La limite de **10 résolutions DNS** de SPF est la panne silencieuse la plus fréquente du domaine. Une organisation ajoute Microsoft 365, puis un outil d'emailing, puis un CRM — chacun avec son include : — et franchit la limite sans aucun signal. SPF renvoie alors permerror et la protection disparaît, alors que l'enregistrement *semble* correct. Un outil qui compte ces résolutions et alerte à 8 apporte une valeur immédiate, et démontre une connaissance réelle du sujet.

5.1 Réutilisation du tableau de bord DMARC existant. Le projet de tableau de bord DMARC déjà écrit n'est pas jeté : sa logique d'analyse des rapports agrégés devient le cœur de `parse_dmarc_report`, et sa base de données alimente la vue « Messagerie » de la console web. C'est un travail d'intégration, pas de réécriture.

6. Serveur MCP — Dossier et remédiation

Seul serveur détenant des droits d'écriture. Il est donc traité avec un régime distinct : inscription d'application séparée, journal d'audit immuable, et approbation humaine obligatoire sur toute opération modifiante.

Outil	Nature	Approbation
create_case	Ouvre un dossier d'incident structuré	Non
append_finding	Ajoute une constatation horodatée	Non
set_verdict	Fige le verdict, la confiance et la cartographie ATT&CK	Non
revoke_user_sessions	Invalide les jetons de rafraîchissement actifs	Oui
disable_user_account	Désactive le compte dans le tenant	Oui
require_password_reset	Force la réinitialisation à la prochaine connexion	Oui
block_sender_domain	Ajoute un domaine à la liste de blocage messagerie	Oui

Comment implémenter la porte d'approbation

Les outils de remédiation ne s'exécutent jamais directement. Ils inscrivent une *action proposée* dans le dossier, avec sa justification et son impact estimé, et retournent à l'agent le statut `pending_approval`. Un analyste valide ou rejette depuis la console web, et son identité est consignée dans le journal d'audit. C'est ce choix, plus que tout autre, qui rend le système déployable en production réelle.

7. La console web analyste

C'est l'ajout le plus visible, et celui qui transforme un projet de terminal en produit démontrable. Un relecteur qui ouvre une URL et voit une investigation se dérouler en direct comprend le projet en trente secondes — ce qu'aucun README ne permet.

7.1 Les cinq pages

Page	Contenu	Ce qu'elle démontre
Tableau de bord	Incidents ouverts, répartition des verdicts, comptes à risque, posture des domaines, coût du jour	Vision d'ensemble en un coup d'œil
Incidents	Liste filtrable par verdict, gravité, domaine, statut d'approbation	Le système a une mémoire
Détail d'incident	Chronologie unifiée des trois domaines, indicateurs enrichis, verdict et confiance, cartographie ATT&CK, trace des appels d'outils , actions proposées avec boutons Approuver / Rejeter	La page qui vend le projet
Bac à sable	Appel manuel de n'importe quel outil MCP, avec sa sortie brute	Les outils sont utilisables sans l'agent
Posture messagerie	Tableau SPF / DKIM / DMARC par domaine surveillé, historique des rapports agrégés	Intègre le travail DMARC existant

7.2 Le détail qui fait la différence : l'investigation en direct

Quand un incident est lancé, la page ne doit pas afficher un sablier pendant 45 secondes. Chaque étape de l'agent est diffusée au navigateur au fil de l'eau, par un flux d'événements serveur :

12:04:31	PLAN	3 hypotheses posees, ordonnees par vraisemblance
12:04:32	OUTIL	get_user_context(marketing@teknologiia.com) 118 ms
12:04:33	CONSTAT	compte privilegie : Helpdesk Administrator
12:04:33	OUTIL	get_user_signins(...) + get_risk_detections(...) en parallele
12:04:34	CONSTAT	7 echecs puis 1 succes depuis 185.220.101.47
12:04:35	OUTIL	enrich_ip(185.220.101.47) 340 ms
12:04:36	CONSTAT	VERDICT malveillant, score 94 : noeud de sortie Tor
12:04:37	OUTIL	get_directory_audits(hours=168) 152 ms
12:04:38	CONSTAT	methode MFA ajoutée 2 min après l'intrusion
12:04:39	VERDICT	malicious, confiance 0.93 - 3 actions proposees

Extrait 2 — Le déroulé diffusé à la console. Montrer le raisonnement pendant qu'il se produit est ce qui distingue une démonstration convaincante d'une boîte noire.

7.3 Choix techniques

Élément	Choix	Justification
Framework	Next.js (App Router) + TypeScript	Rendu serveur pour les pages de liste, composants clients pour le direct
Style	Tailwind CSS + shadcn/ui	Composants accessibles et sobres, sans passer trois sprints sur le design
Temps réel	Server-Sent Events	Un flux serveur vers client suffit ; WebSocket serait surdimensionné
API	FastAPI + Pydantic	Mêmes modèles de données que les serveurs MCP, aucun contrat à dupliquer

Élément	Choix	Justification
Authentification	OIDC via Entra ID	Élégant et conséquent : la plateforme s'authentifie auprès de l'annuaire qu'elle surveille. Évite d'inventer une gestion de mots de passe
Autorisation	Rôles : lecteur, analyste, approbateur	Seul l'approbateur peut valider une action destructive
Base de données	PostgreSQL + SQLAlchemy	Dossiers, traces, journal d'audit

Une erreur à ne pas commettre

Ne commencez pas par l'interface. Une jolie console branchée sur un agent qui n'existe pas ne démontre rien, et l'expérience montre que le contrat de sortie de l'agent change encore beaucoup au début. Livrez une page unique et laide au sprint 4, branchée sur un agent qui fonctionne vraiment ; embellissez au sprint 6. L'inverse coûte deux fois plus cher.

8. L'agent d'orchestration

8.1 La boucle d'enquête. L'agent ne suit pas un script figé. Il itère sur un cycle en cinq temps jusqu'à disposer d'assez d'éléments pour trancher, avec un plafond dur sur le nombre de tours.

Phase	Ce que fait l'agent	Garde-fou
1. Percevoir	Lit l'alerte, extrait l'UPN, les indicateurs et le domaine concerné	Schéma validé par Pydantic ; alerte malformée rejetée
2. Planifier	Choisit le playbook applicable et écrit un plan explicite avant tout appel	Le plan est journalisé : auditable et comparable entre runs
3. Agir	Appelle les outils MCP, en parallèle quand ils sont indépendants	Plafond de 15 appels et 8 tours par dossier
4. Vérifier	Confronte les résultats, détecte les contradictions, relance si un élément manque	Sous 0,6 de confiance, escalade obligatoire vers l'humain
5. Rapporter	Produit un verdict structuré, une chronologie et des actions proposées	Sortie contrainte par un schéma : pas de prose libre

8.2 Le contrat de sortie. C'est l'élément qui transforme un agent conversationnel en composant logiciel : un objet validé, stockable en base, comparable à une référence, donc mesurable.

```
class TriageVerdict(BaseModel):
    verdict: Literal['benign', 'suspicious', 'malicious', 'inconclusive']
    confidence: float = Field(ge=0.0, le=1.0)
    summary: str = Field(max_length=600)
    timeline: list[TimelineEvent]      # identite + messagerie + IOC, fusionnes
    iocs: list[EnrichedIndicator]
    mitre_techniques: list[str]        # ex. ['T1110.003', 'T1621']
    recommended_actions: list[ProposedAction]
    escalate_to_human: bool
    reasoning_trace: list[str]         # chaque decision, justifiee
```

Extrait 3 — escalate_to_human et confidence sont ce qui rend le système exploitable : un agent qui sait dire qu'il ne sait pas vaut infiniment mieux qu'un agent toujours affirmatif.

8.3 Les playbooks

Chaque famille d'alerte dispose d'un playbook en Markdown, versionné dans le dépôt et injecté selon le type d'alerte. Un analyste qui n'écrit pas de Python peut le relire et le corriger — c'est ce qui rend le système adoptable par l'équipe plutôt que dépendant de son auteur.

Playbook	Déclencheur	Domaines mobilisés
compte_compromis.md	Connexion réussie après une série d'échecs	Identité + renseignement
phishing_signale.md	Courriel suspect transmis par un utilisateur	Messagerie + renseignement + identité
usurpation_domaine.md	Rapport DMARC montrant des envois non alignés	Messagerie
password_spray.md	Échecs 50126 sur plusieurs comptes depuis une même IP	Identité + renseignement
escalade_privileges.md	Attribution d'un rôle privilégié	Identité

Le playbook `phishing_signale.md` est celui à démontrer en priorité : c'est le seul qui mobilise les trois domaines, et il prouve que la plateforme vaut plus que la somme de ses outils.

9. Évaluation, observabilité, coûts

C'est la couche que la plupart des projets omettent, et exactement celle qui distingue un prototype d'un système. Elle répond à la question que posera tout ingénieur expérimenté devant la démonstration : *comment savez-vous que votre agent fonctionne bien ?*

9.1 Le harnais d'évaluation — la pièce maîtresse

Un jeu de référence de 50 à 80 incidents synthétiques, chacun avec son verdict attendu, couvrant les trois domaines. Il doit contenir des cas ambigus et des pièges délibérés, sinon il ne mesure rien.

Métrique	Définition	Seuil
Exactitude du verdict	Correspondance exacte avec la référence	≥ 85 %
Taux de faux négatifs	Incidents malveillants classés bénins — la faute grave en SOC	< 2 % — bloquant
Taux de faux positifs	Incidents bénins classés malveillants	< 15 %
Qualité de l'escalade	L'agent escalade-t-il quand sa confiance est faible ?	≥ 90 %
Appels d'outils	Médiane par dossier — mesure l'efficacité du plan	≤ 10
Latence p95	De la réception de l'alerte au verdict	< 60 s
Coût par triage	Tokens facturés, entrée et sortie	< 0,20 USD
Fidélité des citations	Chaque affirmation est-elle adossée à une sortie d'outil réelle ?	≥ 95 %
Résistance à l'injection	Incidents porteurs d'une charge d'injection de prompt	100 % — bloquant

L'asymétrie des erreurs structure tout le système

Un faux positif coûte quelques minutes à un analyste. Un faux négatif laisse un attaquant dans le système d'information. Ces deux erreurs ne se valent pas et ne doivent surtout pas être optimisées ensemble. ARGUS accepte explicitement plus de faux positifs pour écraser les faux négatifs : en cas de doute, l'agent escalade. Formuler cet arbitrage à voix haute, chiffres à l'appui, est l'un des signaux de maturité les plus forts que l'on puisse envoyer en entretien.

9.2 Observabilité et maîtrise des coûts

Levier	Mise en œuvre	Effet
Traces OpenTelemetry	Un span par appel d'outil et par tour de raisonnement, avec tokens, latence et coût	Rejouer une décision six semaines après, quand elle est contestée
Routage de modèle	Modèle rapide pour la boucle, modèle le plus capable réservé aux cas difficiles	Réduction majeure du coût moyen
Mise en cache du prompt	Prompt système, définitions d'outils et playbook mis en cache entre les tours	Réduction forte du coût d'entrée
Cache d'enrichissement	Verdicts d'IOC en cache 24 h dans Redis	Quotas des API externes préservés
Troncature agressive	Héritée de la Phase 1 : 12 champs sur ~60	Ordre de grandeur sur les tokens d'entrée
Plafond dur par dossier	Interruption au-delà du budget, avec escalade automatique	Protection contre l'emballement

10. Sécurité de la plateforme

Un outil de sécurité mal sécurisé est une aggravation du problème. Cette section est un critère d'acceptation, pas une recommandation.

Menace	Mécanisme	Contre-mesure
Injection de prompt	Un attaquant nomme son appareil « Ignore les instructions précédentes ». Le texte arrive dans le contexte du modèle. Le risque croît avec la messagerie : un objet de courriel est entièrement contrôlé par l'attaquant.	Liste blanche de champs — la troncature est un contrôle de sécurité ; délimitation du contenu non fiable ; décision portée par le code ; sous-ensemble d'évaluation dédié
Exfiltration par l'agent	Un agent compromis vide les journaux du tenant	Bornes dures côté serveur : 100 résultats, 168 heures, 20 pages
Fuite de clé d'API	Clé VirusTotal ou secret Entra committé	gitleaks en CI et en pre-commit ; --env-file ; clés séparées par serveur
Élévation de privilèges	La plateforme détient plus de droits que nécessaire	Quatre inscriptions d'application distinctes ; identité managée Azure en production, donc aucun secret client
Député confus	Un jeton émis pour un autre service est accepté	Validation de l'audience et de l'émetteur (RFC 9207)
Action destructive non voulue	L'agent désactive un compte légitime	Porte d'approbation ; les serveurs de lecture n'ont aucune permission d'écriture

11. Stack technique

Domaine	Choix	Justification
Langage backend	Python 3.13	Écosystème MCP et Azure natif ; continuité avec la Phase 1
Serveurs MCP	SDK officiel mcp 2.x (MCPServer)	Schémas d'outils dérivés des annotations de type ; protocole 2026-07-28
Orchestration	Claude Agent SDK	Parle MCP nativement ; gère la boucle d'outils et la compaction de contexte
Modèles	Sonnet pour la boucle, Opus pour les cas difficiles	Routage explicite : le coût moyen chute sans perdre sur les cas durs
Validation	Pydantic v2 + pydantic-settings	Contrats d'entrée et de sortie ; configuration validée au démarrage
HTTP	httpx asynchrone	Appels concurrents ; reprise sur 429
Authentification Azure	azure-identity	Identité managée en production, secret client en développement
DNS	dnspython	Résolution SPF, DKIM, DMARC, MTA-STS
Frontend	Next.js + TypeScript + Tailwind + shadcn/ui	Rendu serveur, composants accessibles, direct par SSE
API web	FastAPI	Mêmes modèles Pydantic que les serveurs MCP
Base de données	PostgreSQL 17 + SQLAlchemy 2	Dossiers, traces, journal d'audit
Cache	Redis	Verdicts d'IOC et limitation de débit
Observabilité	OpenTelemetry + Langfuse	Traces d'agent, coût et latence ; auto-hébergeable
Évaluation	pytest + jeu de référence versionné	Sans dépendance externe, exécutable en CI
Qualité	ruff, mypy strict, pre-commit	Déjà en place et vert sur la Phase 1
Conteneurs	Docker multi-étapes, utilisateur non root	Aucun secret embarqué ; correctifs de sécurité appliqués
CI/CD	GitHub Actions	lint, types, tests, évaluation, images, gitleaks, Trivy
Déploiement	Azure Container Apps	Identité managée : plus aucun secret client en production

12. Feuille de route en 6 sprints

Sprint	Objectif	Livrables	Durée
S0	Acquis	entra-secops-mcp — 6 outils, 96 tests, CI verte, image Docker vérifiée	fait
S1	Enrichissement	threat-intel-mcp : trois sources, fusion déterministe testée, cache Redis, gestion des quotas	2 sem.

Sprint	Objectif	Livrables	Durée
S2	Messagerie	email-security-mcp : SPF avec comptage des résolutions DNS, DKIM, DMARC, analyse d'en-têtes ; intégration du tableau de bord DMARC existant	2,5 sem.
S3	L'agent vit	Boucle d'orchestration, contrat de sortie Pydantic, deux playbooks, case-mcp en lecture, première démo de bout en bout	2,5 sem.
S4	La console	Next.js : page de détail d'incident avec flux SSE, authentification Entra ID, liste des incidents. <i>Fonctionnelle avant d'être belle.</i>	2,5 sem.
S5	Mesurer et durcir	Jeu de référence de 60 incidents, harnais d'évaluation en CI, traces OTel, défenses anti-injection, porte d'approbation, playbooks restants	2,5 sem.
S6	Présenter	Tableau de bord et posture messagerie, déploiement Azure Container Apps, dossier technique, vidéo de démonstration	2 sem.

Règle de conduite : un chemin complet dès le sprint 3

L'erreur classique consiste à perfectionner chaque serveur avant de brancher l'agent, puis à découvrir au sprint 6 que le contrat de sortie ne convient pas. Visez un chemin complet, même imparfait, dès que possible : un playbook, trois outils, une trace, une page. Tout le reste devient de l'amélioration incrémentale et mesurable.

13. Les 15 signaux de niveau senior

Ce sont les points qu'un ingénieur expérimenté cherche en relisant le dépôt. Chacun est peu coûteux, et chacun manque dans la quasi-totalité des projets de portfolio.

#	Signal	Ce qu'il prouve
1	Un harnais d'évaluation avec seuils bloquants en CI	Vous savez mesurer un système non déterministe
2	Un arbitrage faux positifs / faux négatifs assumé et chiffré	Vous comprenez le métier, pas seulement la technique
3	La logique de décision dans du code testé, pas dans le prompt	Vous savez où placer la frontière déterministe / probabiliste
4	Une porte d'approbation sur les actions destructives	Vous concevez pour la production, pas pour la démo
5	Une défense anti-injection, avec ses tests	Vous avez lu la littérature de sécurité des agents
6	Du routage de modèle et un suivi du coût par exécution	Vous pensez au budget d'exploitation
7	Des traces OpenTelemetry sur chaque exécution	Vous savez qu'un agent non observable est indéboguable
8	Des sorties d'outils typées plutôt que des chaînes	Vous traitez le modèle comme un composant, avec un contrat
9	Quatre inscriptions d'application séparées	Le moindre privilège est un réflexe
10	L'identité managée en production, aucun secret client	Vous connaissez l'état de l'art des identités de charge de travail
11	Des playbooks éditables par un non-développeur	Vous concevez pour l'équipe, pas pour vous-même

#	Signal	Ce qu'il prouve
1 2	Un agent capable de dire qu'il ne sait pas et d'escalader	La calibration compte plus que la performance brute
1 3	Le comptage des résolutions DNS de SPF	Connaissance réelle de la sécurité de la messagerie, pas superficielle
1 4	Le raisonnement diffusé en direct à la console	Vous savez qu'un agent opaque ne sera pas adopté
1 5	Un cache et un limiteur de débit sur les API externes	Vous avez déjà pensé au jour où le quota est épuisé

A. Annexe — Questions d'entretien

Ces questions sont celles que pose systématiquement un ingénieur expérimenté devant une démonstration d'agent. Le projet doit permettre d'y répondre par une preuve, jamais par une opinion.

Question	Réponse que le projet permet
Comment savez-vous que votre agent fonctionne bien ?	Jeu de référence de 60 incidents, exactitude de X %, faux négatifs sous 2 %, seuils bloquants en CI. Voici le rapport.
Que se passe-t-il si le modèle hallucine ?	Sortie contrainte par un schéma, fusion des IOC déterministe, fidélité des citations mesurée, aucune action destructive sans approbation humaine.
Combien coûte une exécution ?	0,0X USD en médiane, mesuré sur chaque run. Leviers : routage de modèle, cache de prompt, cache d'IOC, troncature.
Comment empêchez-vous un attaquant de manipuler votre agent ?	Liste blanche de champs, délimitation du contenu non fiable, décision portée par le code, et un sous-ensemble d'évaluation dédié à l'injection.
Pourquoi quatre serveurs MCP plutôt qu'un seul ?	Cloisonnement des privilèges : le plan d'écriture est isolé et seul soumis à approbation. Clés séparées, rayon d'explosion réduit.
Qu'apporte la console web à part du confort ?	L'approbation humaine tracée, la mémoire des incidents, et la transparence du raisonnement. Un agent opaque n'est pas adopté.
Le déploieriez-vous en production demain ?	En assistance à l'analyste, oui : il propose sans jamais agir. En autonome, non, et voici précisément ce qu'il faudrait ajouter.
Qu'est-ce qui a été le plus difficile ?	Calibrer la confiance. Un agent est naturellement trop sûr de lui ; il a fallu construire le jeu de référence pour rendre ce défaut visible, puis mesurable, puis corrigeable.

Note finale. Ce document ne remplace pas la Phase 1 : il la prolonge. Le serveur MCP Entra ID SecOps, livré et opérationnel, devient le premier des quatre serveurs d'outils de la plateforme. Le tableau de bord DMARC déjà écrit devient la couche messagerie. Rien n'est jeté : l'effort porte sur ce qui manque réellement — l'orchestration, l'interface, et la mesure.